Technical Test - Backend Engineer

position at Treasury

Oviek Shagya Ghinulur
Backend Engineer
Repository Github: https://github.com/oviekshgya/treasury
Portofolio: https://whoiam.shagya-tech.my.id/

---

1. write a function

func GO(arr []int) int
if arr contains negative and positive number, should return 0
if arr only have positive number, return the smallest number (except 0)
if arr only have negative number, return the biggest number (except 0)
e.g:
arr = [-1,1], the function should return 0
arr = [-1,-7,-5], then function should return -2
arr = [1,2,1,6], the function should return 3

Answer:

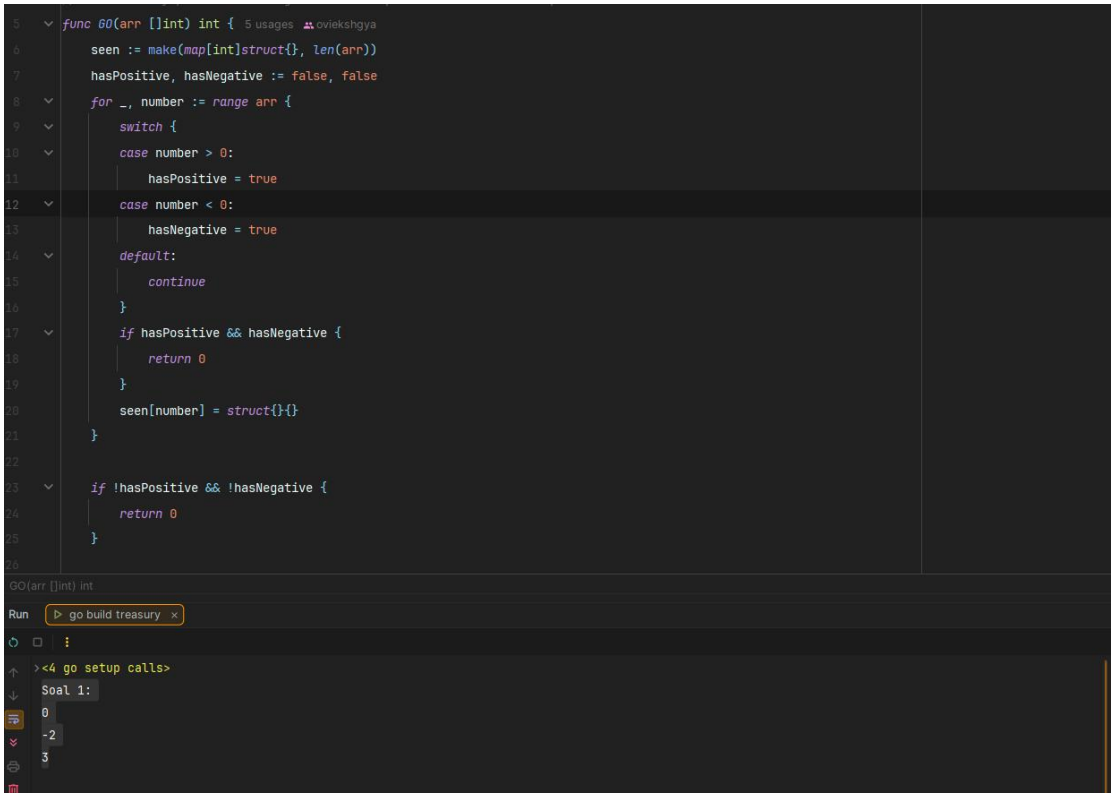
1. Write a function

```
func GO(arr []int) int {
	seen := make(map[int]struct{}, len(arr))
	hasPositive, hasNegative := false, false
	for _, number := range arr {
		switch {
		case number > 0:
			hasPositive = true
		case number < 0:
			hasNegative = true
		default:
			continue
		}
		if hasPositive && hasNegative {
			return 0
		}
		seen[number] = struct{}{}
	}

	if !hasPositive && !hasNegative {
		return 0
	}

	step := 1
	if hasNegative {
		step = -1
	}
	for candidate := step; ; candidate += step {
		if _, exists := seen[candidate]; !exists {
			return candidate
		}
	}
}
```

Output:



Penjelasan:
mencari bilangan yang belum ada: Teks soal dan contohnya berbeda. Jika yang dicari benar-benar nilai terkecil atau terbesar dari isi array, [-1, -7, -5] akan menghasilkan -1, dan [1, 2, 1, 6] akan menghasilkan 1. Namun contoh meminta -2 dan 3. Kode ini mengikuti contoh: mencari bilangan yang belum ada, dimulai dari yang paling dekat dengan nol. Untuk input positif, pencarian berjalan dari 1, 2, 3, ... Untuk input negatif, arahnya -1, -2, -3, ... Angka yang sudah ada dicatat dalam map. Begitu menemukan angka yang belum ada, angka itu dikembalikan. Pada [-1, -7, -5], angka -1 sudah ada dan -2 belum ada, jadi hasilnya -2. Pada [1, 2, 1, 6], angka 1 dan 2 sudah ada, jadi hasilnya 3. Jika positif dan negatif muncul bersamaan, hasilnya 0. Nol diabaikan saat menentukan tanda. Input kosong atau berisi nol saja juga menghasilkan 0.

2. write a function
func GO(exclude []int, nums ...int) (int, error)
join the numbers as one int variable, then do reverse, but exclude number if exists on parameter exclude.
e.q:
GO([]int{0,1,0}, 1000, 1010), the function should return 0, error with message "not found"
GO([]int{0,1,0}, 1259, 2601), the function should return 62952, nil

Answer:

```
// ErrNotFound means no digits remain after applying the exclusion list.
var ErrNotFound = errors.New("not found")

// GO concatenates numbers, reverses the combined number, and removes
excluded digits.
// Minus signs and exclusion values outside 0-9 are ignored.
// No remaining digits returns ErrNotFound; overflow returns
strconv.ErrRange
// wrapped in a *strconv.NumError. Both errors return a zero result.
func GO(exclude []int, nums ...int) (int, error) {
	var excluded [10]bool
	for _, digit := range exclude {
		if digit >= 0 && digit <= 9 {
			excluded[digit] = true
		}
	}

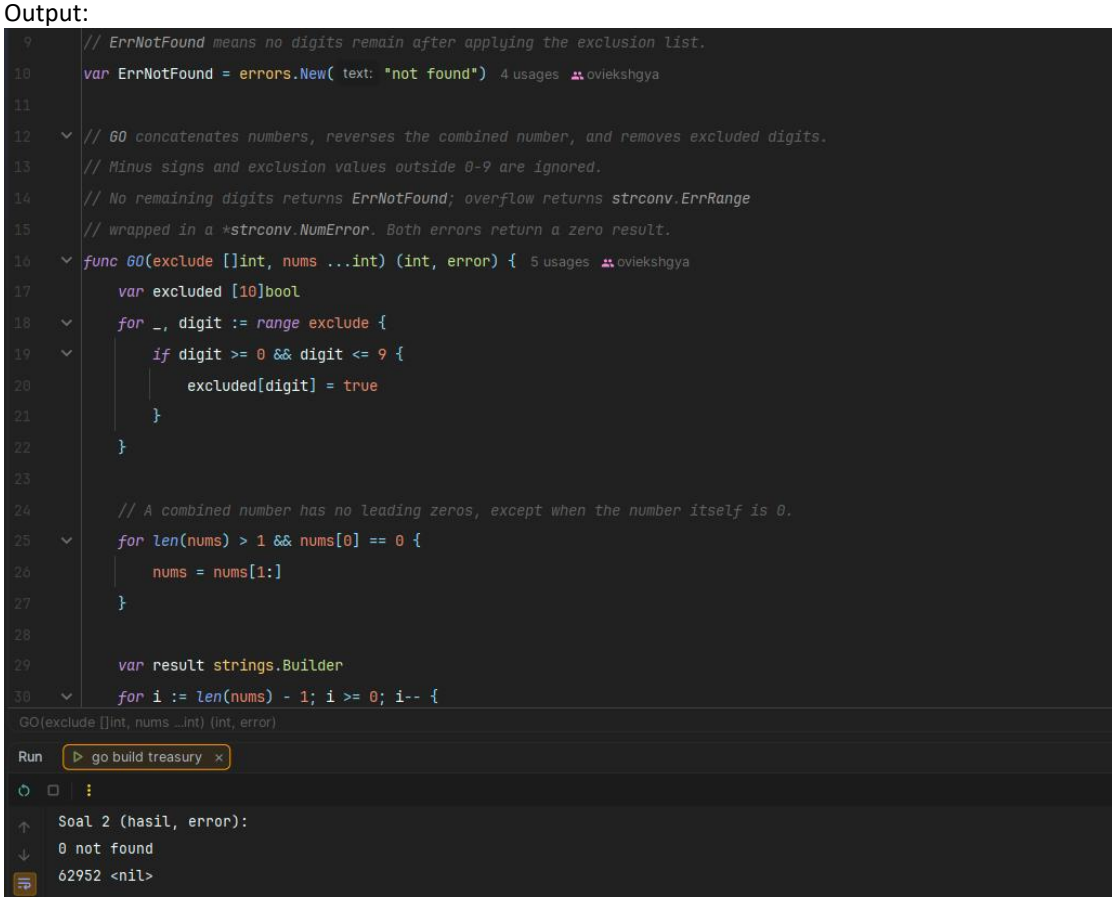
	// A combined number has no leading zeros, except when the number
itself is 0.
	for len(nums) > 1 && nums[0] == 0 {
		nums = nums[1:]
	}

	var result strings.Builder
	for i := len(nums) - 1; i >= 0; i-- {
		digits := strconv.Itoa(nums[i])
		for j := len(digits) - 1; j >= 0; j-- {
			if digits[j] == '-' {
				continue
			}
			if !excluded[digits[j]-'0'] {
				result.WriteByte(digits[j])
			}
		}
	}

	if result.Len() == 0 {
		return 0, ErrNotFound
	}

	number, err := strconv.Atoi(result.String())
	if err != nil {
		return 0, err
	}
	return number, nil
}
```

Output:



Penjelasan:
menggabungkan angka, membalik, lalu membuang digit: Contoh 1259, 2601 dengan exclude berisi 0 dan 1 bisa dibaca seperti ini:

Gabung	12592601
Balik	10629521
Buang 0 dan 1	62952

Kode membaca urutan angka dan digit dari belakang, sambil melewati digit yang dikecualikan. Hasil akhirnya baru dikonversi ke int. Jadi, angka gabungan tidak perlu ditampung dalam int sementara yang bisa lebih dulu mengalami overflow.

Karena gabungan diperlakukan sebagai bilangan, nol di depan tidak dipertahankan. Input 0, 12 menjadi 12, lalu dibalik menjadi 21. Nol di tengah tetap dipakai. Jika semua input nol, masih ada satu digit 0 yang bisa diproses.

Jika tidak ada digit tersisa, fungsi mengembalikan 0 dan error dengan pesan "not found". Error ini juga tersedia sebagai soal2.ErrNotFound. Kalau yang tersisa hanya digit nol, hasilnya 0, nil. Jika hasil akhir melebihi kapasitas int, fungsi mengembalikan 0 dan error konversi.

Soal tidak menentukan perlakuan untuk bilangan negatif atau isi exclude di luar digit desimal. Di sini tanda minus diabaikan, begitu juga nilai exclude di luar 0 sampai 9. Nilai yang berulang dalam exclude tidak mengubah hasil.

3. write a function
func GO(numsA, numsB []int) []int
find numbers on numsA that does exists in numsB.
and do order descending.
e.q:
numsA = [1,2], and numsB = [1,3], the function should return [1].
numsA = [1,2,2] and numsB = [1,2,4], the function should return [2,1].

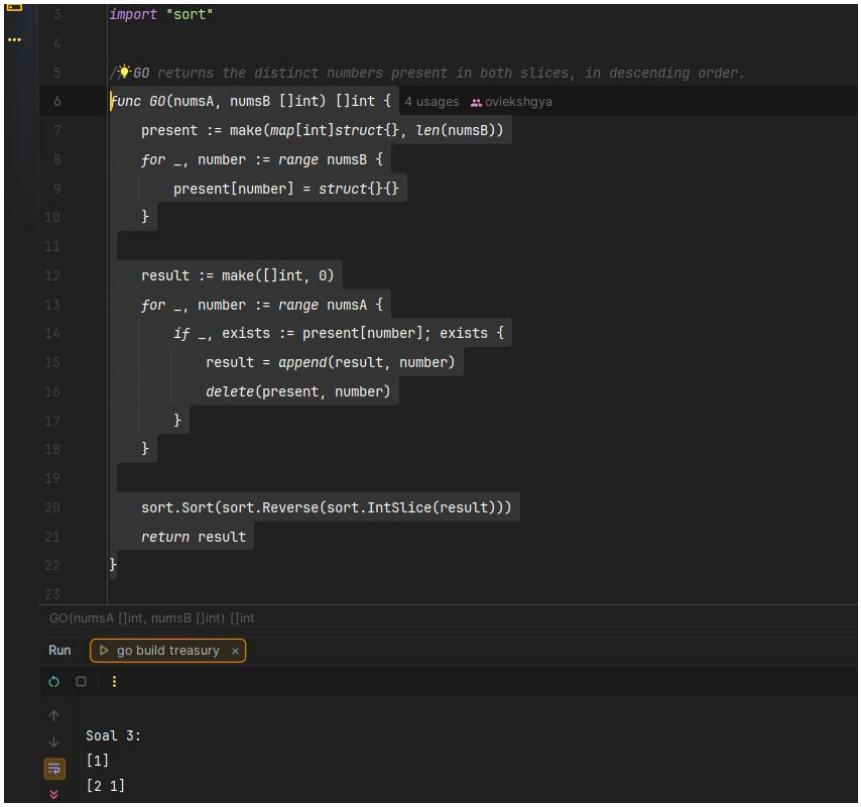
Answer:

```
func GO(numsA, numsB []int) []int {
	present := make(map[int]struct{}, len(numsB))
	for _, number := range numsB {
		present[number] = struct{}{}
	}

	result := make([]int, 0)
	for _, number := range numsA {
		if _, exists := present[number]; exists {
			result = append(result, number)
			delete(present, number)
		}
	}

	sort.Sort(sort.Reverse(sort.IntSlice(result)))
	return result
}
```

Output:



Penjelasan:
mencari angka yang ada di kedua slice: Isi numsB dicatat dalam map, lalu setiap angka di numsA diperiksa terhadap map tersebut. Angka yang cocok dimasukkan ke hasil, lalu dihapus dari map supaya tidak terambil dua kali. Setelah itu hasil diurutkan dari besar ke kecil.

Itulah sebabnya [1, 2, 2] dan [1, 2, 4] menghasilkan [2, 1]. Angka 2 hanya muncul sekali dalam hasil. Jika tidak ada angka yang cocok, hasilnya slice kosong. Kedua slice masukan tidak diubah.

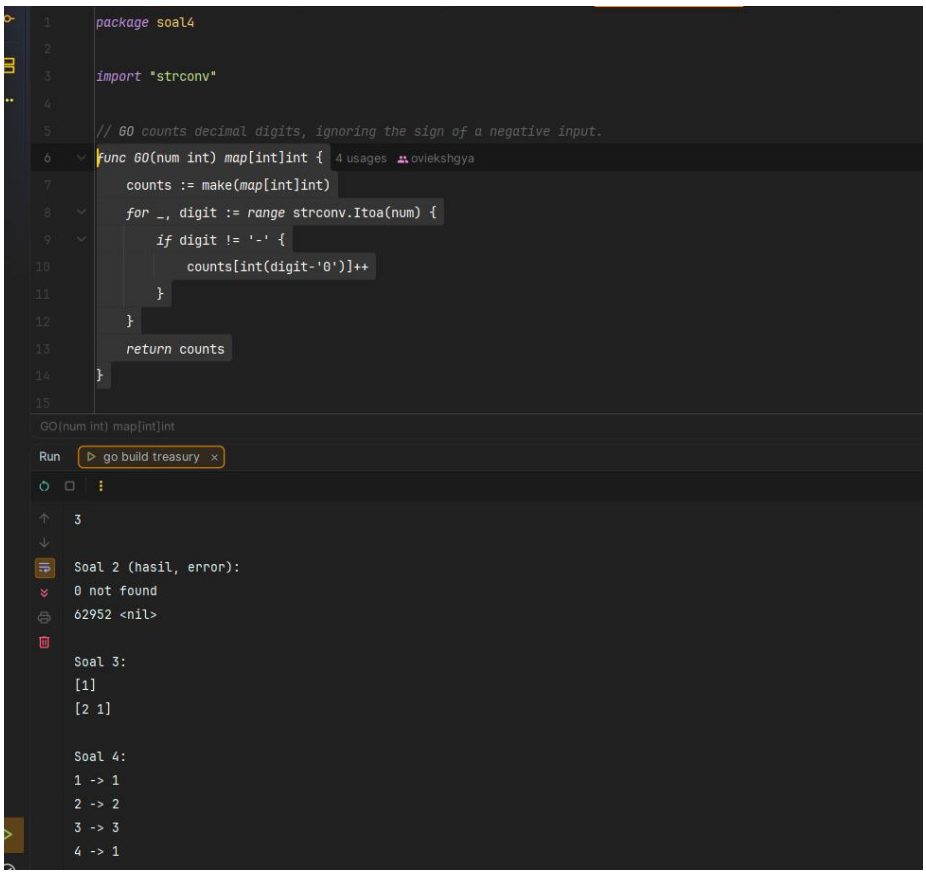
4. write a function
func GO(num int) map[int]int
split the int become array of single digit int, then do group and count the same number.
e.q:
num = 1223334, the function should return
1 -> 1
2 -> 2
3 -> 3
4 -> 1

Answer:

```
func GO(num int) map[int]int {
	counts := make(map[int]int)
	for _, digit := range strconv.Itoa(num) {
		if digit != '-' {
			counts[int(digit-'0')]++
		}
	}
	return counts
```

```
}
```

Output:


package soal4
import "strconv"
// GO counts decimal digits, ignoring the sign of a negative input.
func GO(num int) map[int]int {
4 usages
oviekshgya
counts := make(map[int]int)
for _, digit := range strconv.Itoa(num) {
if digit != '-' {
counts[int(digit-'0')]++
}
}
return counts
}
GO(num int) map[int]int
Run
go build treasury
3
Soal 2 (hasil, error):
0 not found
62952 <nil>
Soal 3:
[1]
[2 1]
Soal 4:
1 -> 1
2 -> 2
3 -> 3
4 -> 1

Penjelasan:
menghitung kemunculan digit: Bilangan diubah menjadi teks, lalu digitnya dibaca satu per satu. Setiap digit menjadi key pada map, dan jumlah kemunculannya menjadi value. Untuk 1223334, hasilnya 1:1, 2:2, 3:3, dan 4:1.

Input 0 dihitung sebagai satu digit nol. Jika input negatif, tanda minus dilewati. Cara ini juga bisa menangani nilai int negatif terkecil tanpa harus mengubahnya menjadi positif terlebih dahulu.

Map tidak menjamin urutan saat dibaca. Karena itu, main.go menampilkan hasil dengan memeriksa digit 0 sampai 9 agar urutannya tetap sama.

5. write a function
func GO(nums ...int) int
do multiplication for 2 number on even index and odd index.
if the last index doesn't have the pair, do multiplication with his own number.
use concurrency to perform the multiplication.
use main thread to perform the summation.
e.g:
nums = [1,2,3,4], the function should return 14, from (1*2) + (3*4), with 2 concurrency process.
nums = [1,2,3,4,5], the function should return 39, from (1*2) + (3*4) + (5*5), with 3 concurrency process.

Answer:

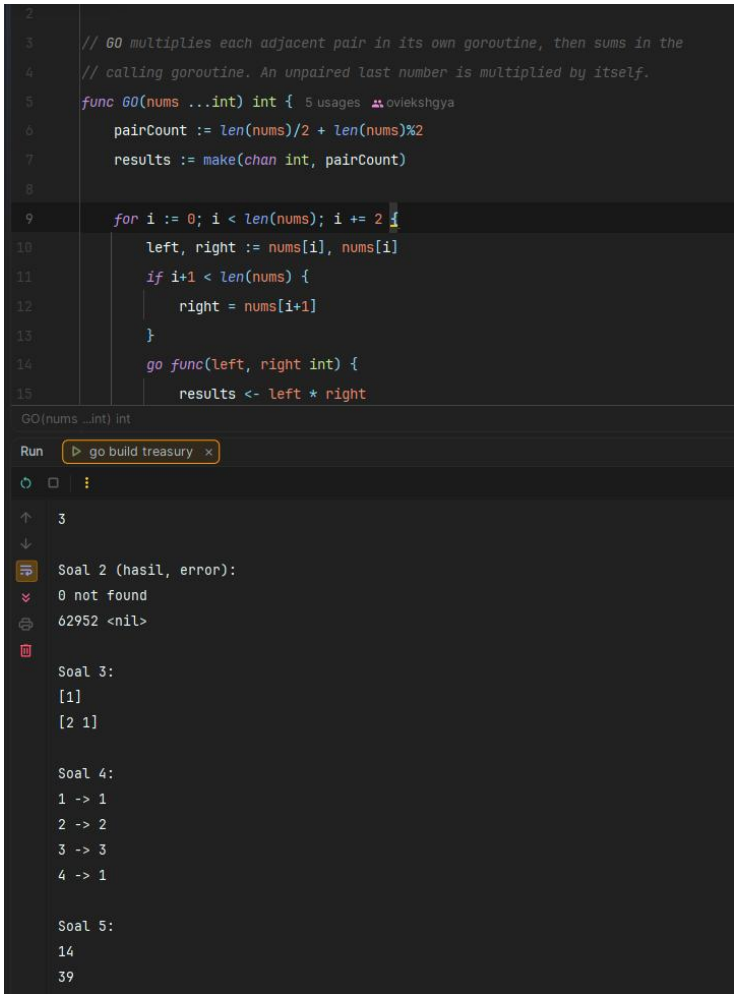
```
func GO(nums ...int) int {
	pairCount := len(nums)/2 + len(nums)%2
	results := make(chan int, pairCount)

	for i := 0; i < len(nums); i += 2 {
		left, right := nums[i], nums[i]
		if i+1 < len(nums) {
			right = nums[i+1]
		}
		go func(left, right int) {
			results <- left * right
		}(left, right)
	}

	total := 0
```

```
	for i := 0; i < pairCount; i++ {
		total += <-results
	}
	return total
}
```

Output:



Penjelasan:
perkalian dengan goroutine: Angka dipasangkan berdasarkan indeks: 0 dengan 1, 2 dengan 3, dan seterusnya. Setiap pasangan dikerjakan oleh satu goroutine. Jika angka terakhir tidak punya pasangan, angka itu dikalikan dengan dirinya sendiri.

Untuk [1, 2, 3, 4, 5], ada tiga goroutine yang menghitung 1*2, 3*4, dan 5*5. Masing-masing mengirim hasil ke channel. Goroutine pemanggil menerima ketiganya dan menjumlahkan 2 + 12 + 25, sehingga hasil akhirnya 39. Saat dipanggil dari main.go, penjumlahan ini berlangsung di goroutine utama.

Urutan hasil masuk ke channel bisa berbeda, tetapi jumlah akhirnya tetap sama. Fungsi menunggu satu hasil untuk setiap pasangan sebelum mengembalikan total. Input kosong menghasilkan 0. Perkalian dan penjumlahan memakai tipe int biasa, tanpa pemeriksaan overflow.